

```

import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
import plotly.express as px

st.title("Mall Customer Segmentation")

# File upload
uploaded_file = st.file_uploader("Upload CSV file", type=['csv'])

if uploaded_file is not None:
    # Load data
    df = pd.read_csv(uploaded_file)

    st.subheader("Dataset Overview")
    st.write(f"Shape: {df.shape}")
    st.dataframe(df.head())

    # Check missing values
    missing = df.isnull().sum()
    if missing.sum() > 0:
        st.subheader("Missing Values")
        st.write(missing[missing > 0])

    # Handle missing values
    strategy = st.selectbox("Missing value strategy", ['median', 'mean', 'drop'])

    if strategy == 'drop':
        df = df.dropna()
    else:
        numeric_cols = df.select_dtypes(include=[np.number]).columns
        imputer = SimpleImputer(strategy=strategy)
        df[numeric_cols] = imputer.fit_transform(df[numeric_cols])

    # Select features for clustering
    numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
    # Remove ID columns
    numeric_cols = [col for col in numeric_cols if 'id' not in col.lower()]

    selected_features = st.multiselect("Select features", numeric_cols, default=numeric_cols[:2])

    if len(selected_features) >= 2:
        # Prepare data
        X = df[selected_features]
        scaler = StandardScaler()
        X_scaled = scaler.fit_transform(X)

        # Elbow method
        st.subheader("Elbow Method")
        max_k = st.slider("Max clusters", 2, 10, 8)

        if st.button("Run Analysis"):

```

```

# Calculate inertias
k_range = range(1, max_k + 1)
inertias = []

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_scaled)
    inertias.append(kmeans.inertia_)

# Plot elbow
fig, ax = plt.subplots()
ax.plot(k_range, inertias, 'bo-')
ax.set_xlabel('Number of Clusters (k)')
ax.set_ylabel('Inertia')
ax.set_title('Elbow Method')
st.pyplot(fig)

# Find optimal k (simplified)
deltas = np.diff(inertias)
optimal_k = np.argmax(deltas[:-1] - deltas[1:]) + 2
st.write(f"Suggested optimal k: {optimal_k}")

# Perform clustering with optimal k
kmeans = KMeans(n_clusters=optimal_k, random_state=42)
clusters = kmeans.fit_predict(X_scaled)

# Add clusters to dataframe
df_clustered = df.copy()
df_clustered['Cluster'] = clusters

# Plot clusters
st.subheader("Cluster Visualization")

if len(selected_features) >= 2:
    fig = px.scatter(
        df_clustered,
        x=selected_features[0],
        y=selected_features[1],
        color='Cluster',
        title=f'Clusters ({optimal_k} clusters)'
    )
    st.plotly_chart(fig)

# Cluster summary
st.subheader("Cluster Summary")
cluster_summary = df_clustered.groupby('Cluster')[selected_features].mean()
st.dataframe(cluster_summary)

# Show cluster sizes
cluster_counts = df_clustered['Cluster'].value_counts().sort_index()
st.bar_chart(cluster_counts)

else:
    st.info("Please upload a CSV file to begin analysis")

# Sample data option
if st.button("Use Sample Data"):

```

```
np.random.seed(42)
n_samples = 200

sample_df = pd.DataFrame({
    'Age': np.random.normal(35, 10, n_samples),
    'Annual_Income': np.random.normal(50, 20, n_samples),
    'Spending_Score': np.random.normal(50, 25, n_samples)
})

# Add some missing values
sample_df.loc[sample_df.sample(frac=0.05).index, 'Age'] = np.nan

st.session_state['sample_data'] = sample_df
st.experimental_rerun()
```